

Programming Assignment 2:

COEN 352-V

Date of Submission: 10/04/2026

Clement Lepage - 40308494

Jules Wygas - 40312026

Introduction:

This programming assignment aims to implement the player behavior for a game of “Forager”. It is a race between two players, placed into a map, who must rush to collect treasures dispersed around said map. Treasures are valued at either 10 or 25 points. Whichever player holds the most points after 50 rounds wins the game.

The assignment requires us to decide on a method to execute the best possible move every round for our players. In other words we need to be able to have our players make as informed a decision as possible in order to increase their chances of victory.

The assignment description provides us with a hint for a method called A*. This algorithm essentially works by evaluating each possible move on the path from start to finish by assigning a ‘g’, ‘h’, and ‘f’ score to each move. As more moves are explored and shorter paths are discovered, previously assigned scores are updated, which in turn may hand us the shortest path from start to finish.

However, we believed that A* wasn’t ideal for this problem as our goal isn’t to get as quickly as possible to the nearest treasure, but rather collect large amounts of treasure, and more importantly, more treasures than the opponent. To that effect, we took inspiration from other game’s computers, especially chess engines, which rely on a search and evaluate method called minimax.

The evaluation method takes the game state, including the player scores, their positions, and all the treasures positions and values. It then evaluates how good that position is for the player. The search function is recursive, at its base case it returns the evaluation of a position, for any higher depth, it returns the maximum of searching at a shallower depth. This entails creating a recursive search tree where a node is a position, and its children are valid moves from that position. See the following pseudo code for a simple explanation of the method:

```
float eval(gamestate)    return Player1_points;

float search(gamestate, depth)
    if(depth == 0 ) return eval (gamestate)

    bestEval = -infinity
    for each move in legal_Moves(gameState)
        gamestate.applyMove
        moveEval = search (gamestate, depth-1)

        if (moveEval > bestEval) bestEval = newEval
    return bestEval
```

Calling this code from each neighbor of the given state will indicate which is best.

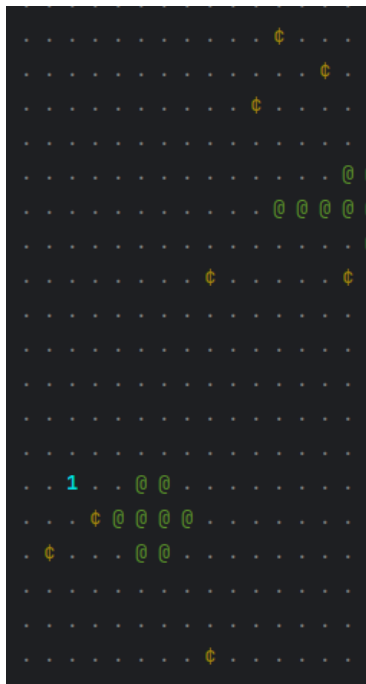
Player 1:

Theory of high depth:

What is the utility of high depth in comparison to a lower depth. A depth of 12 as we use is able to detect clusters of treasures and ignore a near treasure that leads nowhere, here is an example:



Here, a depth any lower than 12 would not be able to detect that the top group contains 2, and would get stranded on the right side.



After picking up the nearest 2, a shallower depths search, or A* to nearest treasure algorithm, would pick up the bottom treasure and be very stranded on the edge of the map. Our solution at depth 12 can detect the upper treasure and takes into account the nearby treasures from there, so it decides to go up and is much more successful for doing so.

Simulation:

First of all the biggest issue with the code snippet in the intro is that we don't have an `applyMove` function with our `gameState`, and we don't want to implement one as that would mean we need to undo it after, so the state can be reused (otherwise it would no longer be a simulated decision). The solution we came up with, after trying the `undo moves` option, was to create a simulated state that we apply the move to. `Sim` takes the previous values of the player's position, and points.

Another issue is updating the knowledge of the treasures in the recursive calls as we don't want the program thinking it can pick up the same treasure twice. Initially we passed a 2d boolean array of `removedTreasures` with the search function so we made sure treasure existed on the map and hadn't been picked up. However, it was more effective to use a `treasures[]` array of booleans that's computed once in `moveDecision` and updated before each recursive call and added back in if that move picked up a treasure. We use a flattened array for better efficiency with a helper function `XYtoI` that converts map coordinates into an array index.

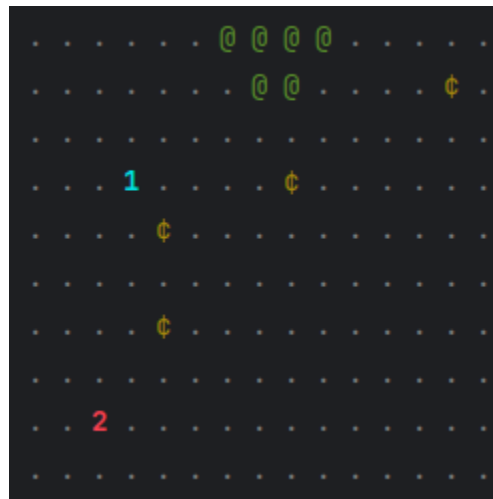
Blindly following P2:

We later noticed that later in the games, when fewer treasures were remaining, our player sometimes followed the other player as they were targeting the same treasure and since they were always behind, the one in front got everything. As a simple remedy, in `moveDecision`, we remove the treasure closest to `Player2` if it's closer to player 2 than to us (we wouldn't want to stop chasing after a treasure we can reach first). Below are some examples of positions where this helps:



On the left, after P2 has collected the third treasure, our optimization forces P1 to ignore the leftmost treasure and it takes the upwards path and manages to collect the top 2 treasure.

Below is a simpler example where the optimisation helps. After P1 collects a treasure it goes for the rightward one even though it's further, as P2 will reach the lowermost treasure first.



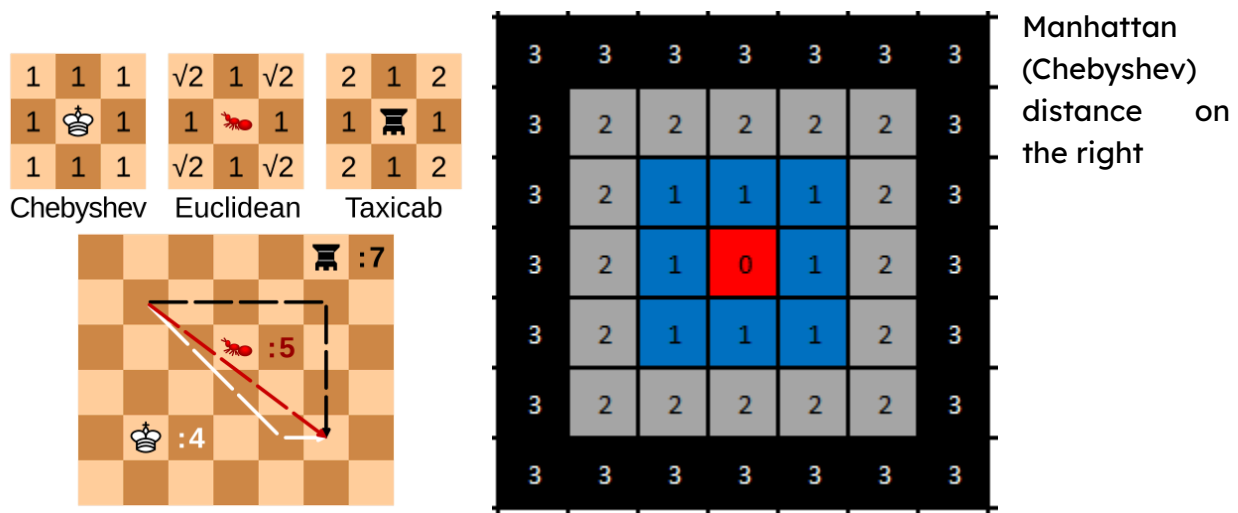
Evaluation and Distance:

The most basic evaluation function choice is Player1's points, but we want to include a term to get it to go closer to treasures in the event that no treasures are in reach. With a high enough depth, this term should be relied upon if the map is scarce in treasure.

We started by checking the Manhattan (separation X + separation Y) distance to every treasure and took the lowest score. However, manhattan doesn't fit our movement exactly as our little foragers can take diagonals in a single move. To take this into account, we used Chebyshev distance, referred to in the project as Maxhattan as it takes $\text{Max}(\text{separation X}, \text{separation Y})$.

The largest issue with our function now was its efficiency. Our implementation iterated through the 2D tiles array from the gameState and calculated their distance and found the minimum. Many treasures are very far and should not be considered. Our solution to this was a square based approach that checks around the player in a larger and larger

square and stops at the first treasure found. Instead of always having to do $80 * 40$ distance calculations, it now had to do an average of 80 checks.



Return by array:

Initially, we had the `moveDecision` function call `search` on each of its neighbors, and return the tile for which the search returned the highest score. The issue was we needed neighbor and simulation logic in the `moveDecision` function which was more messy, but still needed this root node to return a tile, while all its children and all other nodes in the search tree needed to return evaluations. The first solution tried was to make a container object that held both the best tile and evaluation, and was returned in `search()` but this took too much memory. When researching online about how to return a second value in a java function we saw the solution of storing it in an array passed into the function, as shown in the following code block:

```
public int processData(int[] outputArray) {
    outputArray[0] = 100; // This modifies the original array
    return 1; // Standard return value
}
```

A `Tile[] bestTileHolder` and `boolean isRoot` was added to the search function's parameter. The initial function is called with the array we want and `true` for `isRoot`, it recursively calls all the way down the tree with `isRoot = false` and `null` for the array. The root writes its best finding in `bestTileHolder` and `moveDecision`:
`return bestTileHolder[0]`

Alternating and Tie breaker

An issue we had was the fact only best evaluation of a node's subtree is returned to its parent, meaning a move that increases the evaluation instantly (by picking up a treasure) and then stagnates gets the same score as a decision that doesn't do much and picks up a treasure at maximum depth. To remedy this, for the root node, we add a small bonus for moves that increase the evaluation right away, like picking up a treasure:

```
if (isRoot) {  
    neighborSearchEval += 0.1f * eval(simulation, treasures);  
}
```

This had to be balanced as a too high weight would ruin the value of a high depth search, and its ability to discover a path that leads to a high reward in 8 to 10 moves.



The image on the left shows a situation where the three tiles on the left of P1 are all 4 steps away from the goat treasure according to `maxhattan()`.

The tie breaker also fixes this case as the

Optimisation (diagonals and empty maps):

For space and time efficiency purposes we made one decisive change in our search method. We started using an `arrayList` for 'tiles of interest' to store tiles we would like to explore. Based on the fact that diagonal movements cost the same as lateral/vertical displacements, and can lead to more destinations, diagonal movement is almost always the optimal choice. Hence why the usual case has our player adding all 4 diagonals to the tiles of interest array to search from there on, with some notable exceptions.

We had to consider certain specific scenarios. Such as when we had a treasure in our immediate vicinity, but not on a diagonal. In which case we would add that tile to tiles of interest.

Additionally, certain maps have our players faced with many obstacles and tunnels. Hence why we began counting the number of collision tiles in our neighbors array. If that number equaled and/or exceeded 4, we would add all 8 immediate tiles to the tiles of interest array, for the collision tiles to be skipped over when iterating through tiles of interest. This would allow for us to consider however many valid moves are available to us, regardless of whether they are a diagonal or not.

In other few cases, the players would collect all treasures in the map before the end of the round. This would lead to extensively long, and wasteful search times. A simple issue which we resolved with a 'return null' statement in the event that no treasure was found in the map. We used a previously calculated distance to the nearest treasure from the removal of p2's nearest (see: Blindly following P2).

Player 2:

Player 2 has the exact same implementation as player 1, but we set it to depth 6 to show that this still produces satisfactory results. When running matches against each other, higher depth still outperforms lower depth. Hence supporting our claim that high depth simulation is an apt method for player movement decision.

Performance Assessment:

When testing we noticed several behavioral issues in our players which needed to be addressed. Certain scenarios combined a set of circumstances (player/treasure spawn, map collision tile placement) which led to both players following each other. In effect they were both chasing after the same treasure, which meant that whichever player got to the single nearest treasure first would win every time. This led us to exclude the closest treasure to the opponent as a viable direction, removing it from treasures[] (see: Optimisations) .

Later on, when testing our code on other maps (specifically 'Tunnels'), we noticed that our players could get stuck in tight spaces when faced with little choice for movement. Due to our choice to selectively pick which neighboring tiles were considered 'tiles of interest', facing almost only collision tiles led to equally valued choices, which led to the player remaining stuck. To remedy this issue, we elected to verify the number of collision tiles amongst the neighbors, and add all neighboring tiles as tiles of interest if the number of neighboring collision tiles equaled, or exceeded 4 (out of 8). Our implementation already filtered out any collision tiles if they were considered, meaning we would in these cases only consider all non-collision tiles for movement.

In the early testing stages, we observed some very interesting but perplexing results when running the program with shallower depths. Players with smaller depths would sometimes win against their opponent, though they were operating with a higher depth search. We believe this may be due to a combination of factors. Treasure/player spawn, map layout. After testing with swapped spawn points and deterministic treasure spawns, higher depth was shown to win more consistently.

To benchmark our progress in optimisation, we kept track of the maximum depth we were able to run the program at for it to run in under 2 seconds/move.

Max depth	Average positions searched (open area)	Improvement made
6	8^6 250,000	Initial flattened treasure array
8	8^8 16,000,000	Square by square treasure detection
12	4^{12} 16,000,000	Tiles of interest at all depths Same volume of positions, much higher depth

Time and space complexities:

Our search function creates a search tree where each node has between 2 and 5 paths to consider. Since we traverse it in a breadth-first manner, our search time complexity is exponential. On average, 4 tiles of interest are considered at each node, giving the complexity:

$$O(4^n)$$

Search doesn't use much space, with the only growing term being the copies (simulation) of gamestate we make at each new depth, and overwrite for tiles of interest, giving the linear term:

$$O(n)$$

For our nearest treasure function, the number of treasure checks grows quadratically with a maximum upper bound of 50, but it finds a treasure in 5 moves on average.

Theoretically: $O(n^2)$ In practice: $O(1)$ Space complexity: $O(1)$

Reflection:

Our method relies on evaluating moves based on how good a position a certain move takes us to, and how good a score we can obtain following that first move. Hence the more decisions we simulate per turn (the higher the search depth), the more successful the end result ought to be. Considering the nature of this game, we began the assignment with the belief that having a deeper evaluation of our player's move was more valuable (for our method), than taking an opponent's move into consideration. Hence why we use `moveDecision()` in the company of several helper functions (e.g `eval()`, `search()`, `XYtoI()`) to properly replace `findPath()`, and `predictPath()`.

The clear separation in design using the three functions provided in the starter code are a clear fit for the A* algorithm implementation. However, seeing as our team went in the direction of search trees, we elected to use one main recursive search function, and an evaluation function, which would suit our design better.

The maps' designs brought about necessary exceptions for us to take into consideration when implementing player movement decisions. For example the first version of the diagonal movement optimisation had our player getting stuck in narrow parts of the map. With equally evaluated tiles in its surroundings, and a single direction towards the nearest treasure, the player could remain stuck for the remainder of the game (which is obviously not ideal). We therefore had to modify our implementation to address this issue more generally (as to apply in a similar manner on all maps).

In previous versions of our code, we attempted to reach even higher search depths by pruning from our simulations, positions which we could infer would be less fruitful. This proved to be very difficult for multiple reasons. In itself, it was not easy to design a recursive function which would additionally account for pruned positions. Furthermore we could not dedicate too much time finding an optimal way to decide whether a position was worth exploring or not (i.e. picking which positions are pruned, and which are taken into account). This is much simpler in 0 sum games where a deeper evaluation doesn't guarantee a better evaluation, but since our evaluation is focused on our player, it is much more difficult to set a static threshold for what a 'bad move' is.

Our current implementation could in theory be further optimized by attempting to predict our opponent's moves. This however would require more time to imagine the design, and weave it in with our current code.

If we were to continue the project, we would dedicate proper time to think of a way to effectively predict opponent behavior, and use that in collaboration with our current depth search method. The aim would be to find a balance between the two methods to increase our player's success rate without significantly impacting space, and time

complexities. Furthermore, we would address code organization. We would compartmentalize our code more effectively, to have more files in total, but have them be more concise. This would help with code readability, as well as our ability to modify specific parameters, and behaviors with ease.

Conclusion:

This report breaks down the posed objective of this assignment. To design and implement the methods for player movement in a game of foragers, a suggested path for us to follow to that end, as well as the choices we made in implementing our own method and logic. We chose not to go down the path of the A* algorithm as we found it didn't best meet the requirements of the assignment. For it to do so better we would've had to modify some of its concepts to best fit our needs. Seeing as we were beginning with no player movement, we thought it just as well to come up with an alternative, which led to us drawing inspiration from chess search and evaluation methods for our players.

We extensively explain the logic and design choice behind our player movement implementation. From our distanceToNearestTreasure calculations, to our search and evaluation methods. Furthermore, this report covers the exceptions we had to put in place to handle tricky, but recurring situations, such as tight movement (tunnels), or players following each other.

This report also outlines the challenges, and issues we faced along the way, as well as our approach to move past them, until we've reached our current version which responds to the total needs of the assignment.

We are however aware that there is plenty of room to improve, and optimise. We mention that code organization and upgradability is a matter we would address if we were to continue this project, as well as weave in opponent player prediction into our current implementation to improve player win rate.

References:

<https://en.wikipedia.org/wiki/Minimax> - Adapted for single player

https://en.wikipedia.org/wiki/Chebyshev_distance - Maxhattan distance

https://en.wikipedia.org/wiki/A*_search_algorithm - Not used but researched